FindIt

MongoDB Database Architecture

Collections · Fields · Data Types · Constraints · References · Indexes

Technology: MongoDB + Mongoose **Application:** React + Node.js + Express

This document is the database blueprint for the FindIt Lost & Found platform. It describes the planned collections (similar to SQL tables), fields (similar to columns), data types, validation rules, unique constraints, references (similar to foreign keys), and indexes.

Important: MongoDB is a document database, not a relational SQL database. Each document has an _id primary identifier. Related documents can store another document's ObjectId as a reference, but MongoDB does not automatically enforce SQL-style foreign keys. The application must validate references and permissions.

Version 1.0 · Proposed design for the learning project

1. Database overview

Database name: findit
Database type: MongoDB document database
ODM: Mongoose (Node.js)
Primary identifier: ObjectId in the _id field.

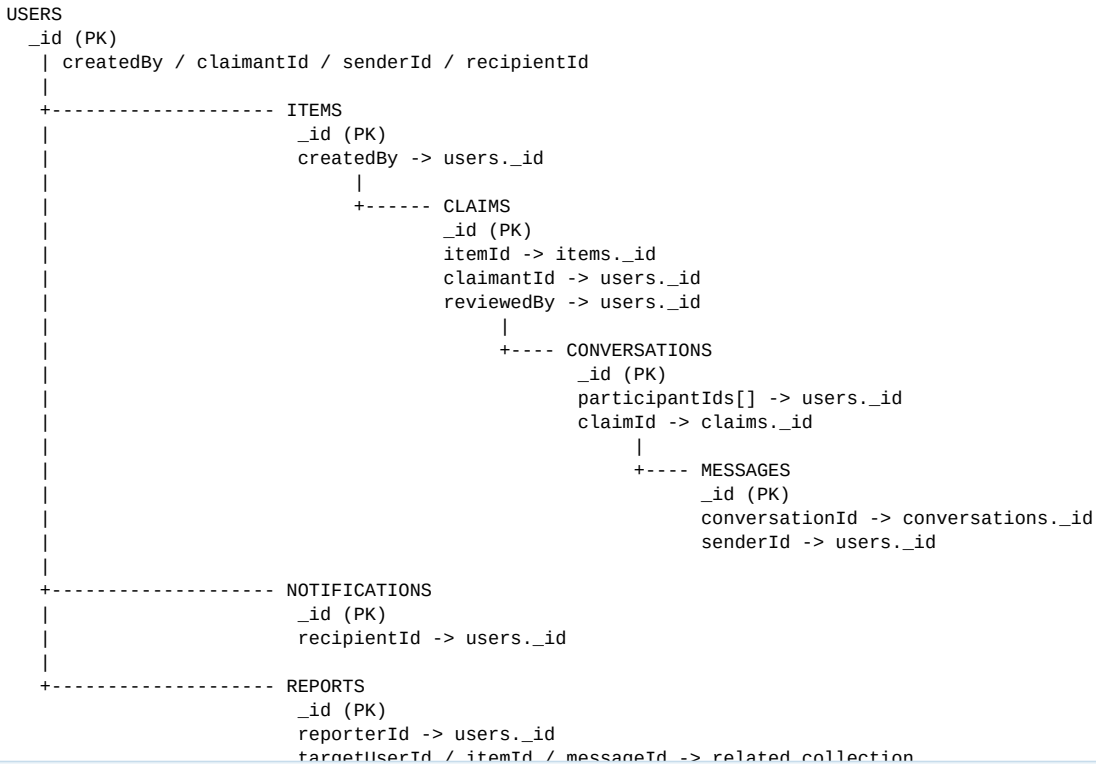
Collection	Purpose
users	User accounts, login identity, roles and account status.
items	Lost-item and found-item posts.
claims	Ownership requests submitted against found items.
conversations	Private chat threads between permitted participants.
messages	Individual messages inside a conversation.
notifications	Updates delivered to a user.
reports	Reports about suspicious posts, users or messages.

Your current setup: You have created the findit database and a users collection in MongoDB Compass. This document describes the proposed complete design. The remaining collections and fields will be implemented gradually. An empty collection does not automatically contain predefined columns or fields.

How to read the schema

Term	Meaning
PK	Primary key. In MongoDB this is normally _id.
REF / FK-like	ObjectId reference to another collection. It is not an automatically enforced SQL foreign key.
Required	Must be supplied and pass validation.
Unique	Duplicate values are rejected by a unique index.
Enum	Only one of the listed values is allowed.
Default	Value applied when the application does not provide one.
Optional	May be absent or null, depending on schema rules.

2. Collection relationship architecture



Relationship summary: one user can create many item posts; one found item can receive many claims; a claim may have a conversation; a conversation contains many messages; users can receive many notifications and submit many reports.

Design decisions

- Lost and found posts share one items collection; type is either lost or found.
- A claim refers to a specific found item. A lost post can help users search for a match, but a claim is submitted against the actual found-item post.
- Chat participants are stored as an array of user ObjectIds. The backend checks membership before returning messages.
- Reports use targetType plus the matching target reference.

3. users collection

Stores registered accounts and authorisation information.

Field	Data type	Key / constraint	Description
_id	ObjectId	PK; auto-generated	Unique user identifier.
name	String	Required; trimmed	User's display name.
email	String	Required; Unique; lowercase	Login email. Create a unique index.
passwordHash	String	Required; private	bcrypt hash. Never store plain-text passwords or return this field in APIs.
role	String	Enum; default user	Allowed values: user, admin.
accountStatus	String	Enum; default active	Allowed values: active, suspended, disabled.
avatarUrl	String	Optional	Profile image URL.
createdAt	Date	Default: now	Account creation date.
updatedAt	Date	Auto-updated	Last modification date.

Implementation notes: Unique index on email. Use Mongoose timestamps. Never expose passwordHash in API responses.

4. items collection

Stores both lost-item and found-item posts. The type field distinguishes them.

Field	Data type	Key / constraint	Description
_id	ObjectId	PK; auto-generated	Unique item-post identifier.
createdBy	ObjectId	REF -> users._id; Required	User who created the post.
type	String	Enum; Required	Allowed values: lost, found.
title	String	Required; trimmed	Short item name, e.g. Black wallet.
description	String	Required	Public description; do not reveal secret ownership clues.
category	String	Enum; Required	electronics, bags, clothing, documents, accessories, keys, other.
locationName	String	Required	Place name, e.g. College Library.
locationDetails	String	Optional	More specific location; avoid sensitive personal information.
eventDate	Date	Required	Date the item was lost or found, based on type.
imageUrls	Array of String	Optional	Image URLs. Store files in image storage, not directly in MongoDB.
privateClues	Array of String	Optional; restricted	Non-public details for claim review. Never include in public item API responses.
status	String	Enum; default open	open, claim_pending, returned, closed, removed.

Field	Data type	Key / constraint	Description
isRewardOffered	Boolean	Default false	Whether the poster offers a reward.
rewardDetails	String	Optional	Optional reward description.
createdAt	Date	Default: now	Post creation date.
updatedAt	Date	Auto-updated	Last modification date.

Implementation notes: Suggested indexes: {type: 1, status: 1, createdAt: -1}, {category: 1}, {locationName: 1}, and optionally text search on title and description. Only the creator or an authorised admin can edit/remove a post.

5. claims collection

Stores a user's request to claim a specific found item.

Field	Data type	Key / constraint	Description
_id	ObjectId	PK; auto-generated	Unique claim identifier.
itemId	ObjectId	REF -> items._id; Required	The found-item post being claimed.
claimantId	ObjectId	REF -> users._id; Required	User who says the item belongs to them.
answers	Array of objects	Required; validate	Verification answers, e.g. question and answer. Keep private.
status	String	Enum; default pending	pending, accepted, rejected, cancelled, completed.
reviewedBy	ObjectId	REF -> users._id; Optional	Finder or authorised reviewer.
reviewNote	String	Optional; private	Private review or handover note.
createdAt	Date	Default: now	Submission date.
updatedAt	Date	Auto-updated	Last modification date.

Implementation notes: Validate that itemId exists and refers to a found item. A user cannot claim their own found post. Only the finder/creator or admin may review. Prevent duplicate active claims.

6. conversations collection

Stores participants and context for a private chat.

Field	Data type	Key / constraint	Description
_id	ObjectId	PK; auto-generated	Unique conversation identifier.
claimId	ObjectId	REF -> claims._id; Required; Unique	Claim that authorises the conversation.
itemId	ObjectId	REF -> items._id; Required	Found item related to the chat.
participantIds	Array of ObjectId	REF -> users._id; Required	Finder and claimant allowed to access the chat.
lastMessageAt	Date	Optional	Timestamp of latest message for sorting.
createdAt	Date	Default: now	Conversation creation date.
updatedAt	Date	Auto-updated	Last conversation update.

Implementation notes: Create the conversation at the agreed claim workflow stage, for example after a claim is accepted. Verify the requester is a participant on every read/send operation.

7. messages collection

Stores individual messages sent inside a conversation.

Field	Data type	Key / constraint	Description
_id	ObjectId	PK; auto-generated	Unique message identifier.
conversationId	ObjectId	REF -> conversations._id; Required	Conversation this message belongs to.
senderId	ObjectId	REF -> users._id; Required	Authenticated user who sent it.
text	String	Required; max length	Message body. Set a sensible maximum length.
messageType	String	Enum; default text	Initial version: text. Image messages can be added later.
isDeleted	Boolean	Default false	Optional soft-delete flag.
createdAt	Date	Default: now	Message timestamp.

Implementation notes: Index {conversationId: 1, createdAt: 1}. Derive senderId from the authenticated user, not from browser input. Verify conversation membership before reading or sending.

8. notifications collection

Stores updates for users, such as claim decisions and new messages.

Field	Data type	Key / constraint	Description
_id	ObjectId	PK; auto-generated	Unique notification identifier.
recipientId	ObjectId	REF -> users._id; Required	User who receives the notification.
type	String	Enum; Required	claim_received, claim_accepted, claim_rejected, new_message, item_update.
title	String	Required	Short notification heading.
body	String	Required	Short explanatory message.
relatedItemId	ObjectId	REF -> items._id; Optional	Related item.
relatedClaimId	ObjectId	REF -> claims._id; Optional	Related claim.
relatedConversationId	ObjectId	REF -> conversations._id; Optional	Related conversation.
isRead	Boolean	Default false	Whether recipient has read it.
createdAt	Date	Default: now	Notification creation date.

Implementation notes: Index {recipientId: 1, isRead: 1, createdAt: -1}. Only the recipient can read or mark their notification as read.

9. reports collection

Stores user reports about potentially inappropriate or suspicious content.

Field	Data type	Key / constraint	Description
_id	ObjectId	PK; auto-generated	Unique report identifier.
reporterId	ObjectId	REF -> users._id; Required	User who submitted the report.
targetType	String	Enum; Required	Allowed values: item, user, message.
itemId	ObjectId	REF -> items._id; Conditional	Required when targetType is item.
targetUserId	ObjectId	REF -> users._id; Conditional	Required when targetType is user.
messageId	ObjectId	REF -> messages._id; Conditional	Required when targetType is message.
reason	String	Required; validated	spam, inappropriate, scam, harassment, other.
details	String	Optional	Additional context.
status	String	Enum; default open	open, under_review, resolved, dismissed.
reviewedBy	ObjectId	REF -> users._id; Optional	Admin who reviewed the report.
adminNote	String	Optional; private	Internal moderation note.

Field	Data type	Key / constraint	Description
createdAt	Date	Default: now	Submission date.
updatedAt	Date	Auto-updated	Last modification date.

Implementation notes: Schema validation should require exactly the target reference that matches targetType. Only admins can review reports and change moderation status.

10. Recommended indexes

Collection	Index	Purpose
users	{ email: 1 } UNIQUE	Prevent duplicate account emails and speed up login lookup.
items	{ type: 1, status: 1, createdAt: -1 }	Browse open lost/found posts, newest first.
items	{ category: 1, eventDate: -1 }	Filter by category and date.
items	Text index on title and description (optional)	Basic keyword search; validate language and search requirements.
claims	{ itemId: 1, claimantId: 1 }	Find a claimant's requests for an item.
conversations	{ claimId: 1 } UNIQUE	Prevent multiple conversations for one claim.
messages	{ conversationId: 1, createdAt: 1 }	Load messages chronologically.
notifications	{ recipientId: 1, isRead: 1, createdAt: -1 }	Show unread and recent notifications.
reports	{ status: 1, createdAt: -1 }	Admin queue for open and recent reports.

A unique index on claims (itemId, claimantId) would prevent a user from ever claiming the same item again, even after cancelling or rejection. For this project, enforce one active claim per user/item in application logic or use a partial unique index for active statuses.

11. Validation and workflow rules

Area	Rule
Users	Email must be valid and unique. Password must be hashed. Role must be user or admin.
Items	type must be lost or found. eventDate, title, category, locationName and description are required. Only creator/admin may edit or remove.
Claims	Claim must refer to an existing found item. A user cannot claim their own found post. Only finder/creator or admin can accept/reject.
Claim status	Suggested flow: pending -> accepted or rejected; pending -> cancelled; accepted -> completed. Restrict invalid transitions.
Conversations	Only finder and claimant are participants. Check membership for every read/send operation.
Messages	senderId comes from authentication. A message must belong to a conversation the sender participates in.
Notifications	Only recipient can read or mark their notification as read.
Reports	Target reference must match targetType. Only admins can review reports.

For a real deployment, also add request rate limits, secure token handling, input validation, upload restrictions, audit logging for admin actions, and a clear data retention/deletion policy.

12. Recommended implementation order

Stage	Collections	What to build
1. Foundation	users, items	Mongoose connection, models, CRUD APIs, browse/search, basic authentication.
2. Ownership flow	claims	Claim form, verification answers, review actions, status tracking.
3. Communication	conversations, messages	Create chat for an eligible claim, Socket.IO delivery, persistent message history.
4. Updates	notifications	Create notifications for claims/messages; show unread status.
5. Moderation	reports	Report content, admin review queue, moderation actions.

13. SQL terminology compared with MongoDB

SQL concept	MongoDB equivalent	Difference
Database	findit database	Groups stored data.
Table	Collection, e.g. users or items	Stores BSON documents, which can have flexible fields.
Row	Document	A JSON-like BSON record.
Column	Field	Fields can contain nested objects and arrays.
Primary key	_id (ObjectId)	MongoDB automatically assigns _id if omitted.
Foreign key	ObjectId reference, e.g. createdBy	MongoDB does not automatically enforce referential integrity.
NOT NULL	Mongoose required validation / schema validation	Validation must be configured.
UNIQUE	Unique index	Enforces uniqueness across documents.
JOIN	$lookup or multiple queries	Choose references or embedded data based on access patterns.

14. Final notes before coding

This is a proposed version-one architecture, not a claim that every collection or field already exists in Compass. Your current findit database and users collection are the starting point. We will create Mongoose schemas and collections as each feature is implemented.

Recommended first coding task: configure the Node.js backend, connect it to mongodb://127.0.0.1:27017/findit, and create the users and items Mongoose models. MongoDB creates a collection when the application first inserts a document into it (or when you explicitly create it in Compass).

End of database architecture document